

Approach Document

SHL Conversational Assessment Recommender — AI Intern Take-Home Assignment

Design choices

The system is a retrieval-grounded pipeline, not a multi-agent framework: guardrail pre-check → analyzer (LLM call) → router → retrieval/compare → responder (LLM call) → validator. FastAPI was chosen for native Pydantic validation matching the required schema exactly; no database or session store is used since the API is contractually stateless — every `/chat` call re-derives everything it needs from the full message history it's given.

The single most important decision: the LLM never free-types a name or URL. The responder only selects assessment **ids** from a closed list of retrieved candidates; the actual recommendations JSON is assembled afterward directly from our own validated catalog records, then cross-checked against the live catalog again in a final validator. This makes hallucinated names/URLs structurally impossible rather than merely discouraged by prompting, and is the main lever against the hard-eval scoring component.

Retrieval setup

The provided catalog (377 Individual Test Solutions) was normalized into a clean schema: name, url, test_type (single-letter code derived from category tags), description, job_levels, languages, duration. Two real data-quality bugs were caught and fixed by cross-referencing the 10 provided traces against the raw scrape: (1) one product name was corrupted by a stray control character mid-scrape — repaired narrowly for that one record, after a broader “rebuild from URL slug” heuristic was tested against the full catalog and found to corrupt 22 otherwise-correct names (e.g. “.NET Framework 4.5” → “Net Framework 4 5”) and was scrapped; (2) multi-category “report” products collapse to Test Type “D” rather than joining every tag, matching the one confirmed example in the traces.

Retrieval uses Google's hosted text-embedding-004 model via the Gemini API over name + categories + job levels + description, indexed with FAISS IndexFlatIP (exact cosine search — at 377 items, approximate indexing adds complexity with no benefit). The query is built each turn from the LLM-extracted constraint set, not the raw user message, so retrieval benefits from accumulated context rather than just the latest sentence. A local sentence-transformers backend was tried first and remains available in the codebase for self-hosting with more memory — see Evaluation approach below for why the default moved to the hosted API.

Prompt design

Two LLM calls per turn, deliberately narrow in responsibility. The **analyzer** re-derives intent and the full constraint set from the entire conversation history on every call (not a diff of the newest message) — this is what makes “refine” work with zero special-case code: a later message that adds or changes a constraint just merges into the same extraction step every time. The **responder** only selects from a candidate list handed to it and writes the reply text; it cannot introduce new names. A separate prompt handles **compare**, grounded strictly in the specific catalog records resolved for the named assessments (exact match, falling back to fuzzy matching for close variants like “OPQ” vs “OPQ32r”).

A code-level guardrail pre-check (regex patterns for injection, legal advice, general hiring advice, and clearly off-topic content) runs before any LLM call, so the clearest refusal cases are instant and don't depend on the model choosing to refuse gracefully.

Evaluation approach

`scripts/run_eval.py` parses the 10 provided trace files (extracting each trace's user turns and its final markdown table as ground truth), replays the user turns against a live `/chat` endpoint, and computes Recall@10 by comparing predicted vs. expected URLs. All ground-truth URLs referenced across the 10 traces were confirmed to exist in the normalized catalog before any pipeline code was written.

What didn't work / limitation to flag honestly: the assignment states the evaluator caps conversations at “8 turns including user & assistant.” Taken literally as 8 total messages, this is contradicted by the traces themselves — trace C9 has 7 exchanges (14 messages) and C3 has 10. Since these are SHL's own ground-truth examples, that evidence was trusted over the ambiguous wording, and the cap was implemented as 8 exchanges (up to 16 messages), with a hard override: on the last available exchange, the router forces a best-effort shortlist rather than risk ending on an unanswered clarifying question with zero recall.

A more consequential lesson: the original retrieval design used a local sentence-transformers model. That worked locally, but the first real deployment to Render's free tier failed outright with "Ran out of memory (used over 512MB)" — PyTorch's footprint alone is close to that ceiling. Rather than paying for a larger instance, the embedding backend was swapped to a hosted Gemini text-embedding-004 API call, reusing the same key already required for chat completions. This required zero changes to the retrieval logic itself, because it was built from the start against a pluggable Embedder interface rather than a specific backend — the swap was a one-line change at the call site. This is exactly the kind of real deployment constraint that's easy to miss when developing locally and worth designing around up front.

A third limitation: the development sandbox used to build this had no internet access to HuggingFace, so retrieval quality was validated locally with a TF-IDF stand-in (keyword-only) rather than a real embedding model, and the pytest suite uses a deterministic fake embedder for the same reason — both clearly scoped as dev/test-only in code comments. The actual deployment runs the real Gemini embedding API; real Recall@10 numbers should be captured with `run_eval.py` against the live deployment before final submission.

AI tool disclosure

This solution was built in collaboration with Claude (Anthropic), used for architecture discussion, code generation across all pipeline modules, catalog data-quality debugging (including catching and reverting a name-corruption regression before it shipped), and drafting this document. All design decisions and trade-offs above were reviewed and are defensible in the technical interview.